
Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

SWAPD453- Spring 2026

IoT App Devs

Distributed Intelligent Campus IoT Environment

Student#1	Mohaned Atef 202100383
Student#2	Elhusseain Aboulfetouh 202202239
Student#3	Kareem Adel 202200059
Student#4	Mohy Eldeen 202201146

04/20/2026

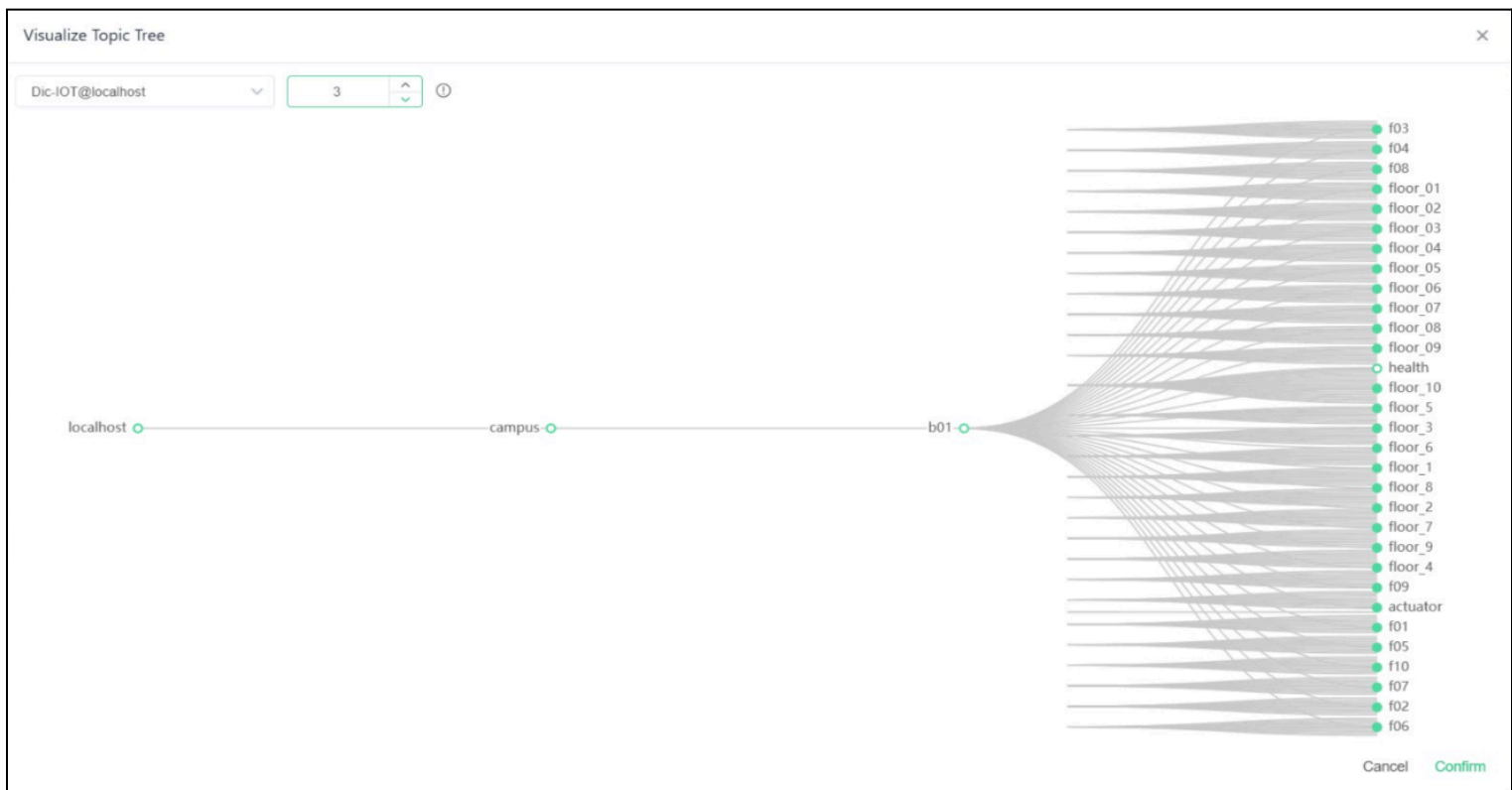
System Overview

This phase architecture relies on a bi-directional loop between two core platforms:

Node-RED: Acts as the data gateway. It collects raw telemetry from 200 field nodes (via MQTT/CoAP), formats the data, and reliably pushes it upward to ThingsBoard.

ThingsBoard: Acts as the central "brain" and dashboard. It visualizes the incoming data in real-time tracking health states, uses its Rule Engine to monitor connection timeouts, and sends remote commands (RPC) back down the chain to control end devices.

The screenshot shows a location-based MQTT topic hierarchy structured as `localhost/campus/b01/[Target]`



Connection State Management:

MQTT Nodes (LWT Implementation): Each MQTT client connects to the broker with a configured Last Will and Testament (LWT) payload. If a node loses its connection abruptly without sending a standard DISCONNECT control packet, the HiveMQ broker automatically fires the LWT message {"status":"Offline"} to the ThingsBoard telemetry/attributes topic. The dashboard instantly reflects this Offline status.

CoAP Nodes (Timeout/Heartbeat Logic): Because CoAP is connectionless (UDP-based), we implemented a Time-To-Live (TTL) Heartbeat mechanism. ThingsBoard is configured with an inactivity timeout threshold (e.g., X = 30 seconds). If "Keep-Alive" or telemetry updates are not received from a CoAP node within this window, the rule engine automatically marks the room's status as "Dead", alerting operators to the communication failure.

Reliability Proof

During the 5-minute stress test, a brief 10-second external network disruption was introduced to simulate poor wireless conditions. Observations:

Handling of the DUP Flag: For MQTT, the network flicker caused intermediate PUBREC packets to drop. As a result, the publishing client retransmitted the PUBLISH packet with the DUP (Duplicate) flag set to 1. Because QoS 2 uses a 4-step handshake (PUBLISH -> PUBREC -> PUBREL -> PUBCOMP), HiveMQ successfully identified the duplicate packet identifier (Packet ID) and safely discarded the duplicate execution, ensuring the command was implemented exactly once.

CoAP CON Retransmissions: The CoAP nodes sent CON messages that initially failed to receive an ACK. The built-in exponential backoff retransmission mechanism kicked in, resending the identical CON packet once the network stabilized. Once the ACK was finally returned, the transaction closed out successfully. This is built in as an option in node red CoAP node.

Edit coap request node

Delete

Cancel

Done

⚙ Properties

⚙

📄

🖼

🔗 URL

coap://

📋 Method

GET

▼

☒ Confirmable?

☒ Observe?

☐ Raw buffer?

☐ Force multicast

🕒 Multicast timeout (in ms)

20000

📄 Content format

application/json

▼

🏷 Name

observe payload

Tip: Leave url or method blank if you want to set them via msg properties.

Below, we configured mqtt to work with qos 2

Edit mqtt out node

Delete Cancel Done

Properties

Server ThingsBoard Direct

Topic v1/gateway/telemetry

QoS 2 Retain

Name Mqtt out

Add new mqtt-broker config node

Tip: Leave topic, qos or retain blank if you want to set them via msg properties.

The "Round-Trip" Latency Benchmark & Audit

To measure the end-to-end responsiveness of the system, tracking Southbound commands to Northbound telemetry updates.

Methodology: The benchmark records the timestamp across three major milestones:

T0: ThingsBoard issues a Command (RPC) / Southbound.

T1: Python Node receives the command and applies the State Change (Virtual Actuator).

T2: Node sends the updated Telemetry back to the Dashboard / Northbound.

Target KPI: Total RTT ($T_2 - T_0$) must be $< 500 \text{ ms}$ under 100% simulation load.

Latency Audit Example data for Hvac_mode:

Device ID	Command Type	Total RTT ($T_2 - T_0$)	Mode	Status
b01-f01-r001	HVAC_ON	299 ms	ON	Success
b01-f01-r001	HVAC_OFF	46 ms	OFF	Success
b01-f01-r015	HVAC_ON	62 ms	ON	Success

b01-f01-r015	HVAC_OFF	46 ms	OFF	Success
b01-f01-r019	HVAC_ON	47 ms	ON	Success
b01-f01-r019	HVAC_OFF	55 ms	OFF	Success